PROGRAMMING FOR ARTIFICIAL INTELLIGENCE LAB MANUAL

Dr. Muhammad Siddique

Programming for Artificial Intelligence Lab Manual (Python using IDLE / VS Code / Google Colab)

Dr. Muhammad Siddique

August 2026

Contents

1 Week 1: Introduction to Python and the Development Environment

Objective: Set up a Python development environment, understand the difference between interactive and scripted execution, and run your first programs.

Tasks:

1. **Task 1:** Install Python (or use Google Colab / Anaconda), launch an interpreter session, and explain what the REPL prompt is used for.
2. **Task 2:** Use the interpreter as a calculator for arithmetic expressions, and explain how integer and floating-point results differ.
3. **Task 3:** Write, save, and execute a script file from an IDE or command line, and describe the edit–run–fix cycle.
4. **Task 4:** Add comments to a program and observe which lines the interpreter ignores, and explain why commenting matters in team-based AI projects.
5. **Task 5:** Print multiple lines and formatted output using `print()`, and explore what happens when you make deliberate syntax errors.

Details of Lab Experiment:

Task 1: The Python interpreter (REPL)

Python is an *interpreted* language: statements are executed one at a time by the Python runtime rather than being compiled ahead of time into machine code. Launching `python` (or opening a notebook cell) drops you into the Read–Evaluate–Print Loop, which immediately echoes the result of every expression – making it ideal for quick experiments with libraries before committing them to a program file.

Task 2: Python as a calculator

```
>>> 2 + 3
5
>>> 10 / 4          # true division always yields a float
2.5
>>> 10 // 4         # floor division discards the
   remainder
```

```
2
>>> 10 % 4           # modulus keeps only the remainder
2
>>> 2 ** 10          # exponentiation
1024
```

Listing 1: Arithmetic in the interactive interpreter

Notice that a single slash `/` always produces a floating-point result even when the division is exact, while `//` performs floor division. These distinctions matter later when computing accuracies and error rates in AI labs, where mixing up integer and float division silently changes results.

Task 3: Your first script

```
# hello_ai.py -- my first Python program
print("Hello, Artificial Intelligence!")

name = input("What is your name? ")
print("Welcome to the world of AI,", name)
```

Listing 2: hello_ai.py – a first Python program

Save the file as `hello_ai.py` and run it either from the IDE's Run button or from a terminal with `python hello_ai.py`. Unlike the REPL, a script executes all its lines from top to bottom without echoing intermediate values – anything you want to see must be printed explicitly. If you mistype a statement, Python stops and reports the line number along with the error type; learning to read these tracebacks is a core skill you will use every single week of this course.

Task 4: Comments

Everything after a `#` symbol on a line is ignored by the interpreter. Comments exist purely for human readers: six months from now you are the most likely reader of your own code, and in AI projects – where data-cleaning steps and model choices must be justified – clear comments are not optional polish but essential documentation.

Task 5: Formatted printing

```
course = "Programming for AI"
marks = 88.56789

print("Course:", course)                    # comma-
    separated
```

```
print(f"Course: {course}")                    # f-string (
    preferred)
print(f"Marks: {marks:.2f}")                  # rounded to 2
    decimals
print("Line1\nLine2")                         # newline escape
```

Listing 3: Multiple ways to format output

The f-string syntax embeds variables directly inside string literals and supports format specifiers such as `:.2f` for fixed decimal places. You will use this constantly to report metrics like `Accuracy: 0.94` in later labs.

Summary:

This week you installed and explored a Python environment, distinguished the REPL from script execution, wrote and ran your first programs, and learned how errors and comments behave. Everything in the remaining twelve labs assumes this foundation is solid.

2 Week 2: Variables, Data Types, and Input/Output

Objective: Store and manipulate data using Python's core types, convert between them, and build interactive programs that read input and produce formatted output.

Tasks:

1. **Task 1:** Declare variables of each core type (`int`, `float`, `str`, `bool`) and inspect them with `type()`.
2. **Task 2:** Convert between types using constructor functions, and identify exactly when an implicit versus explicit conversion happens.
3. **Task 3:** Read keyboard input with `input()` and explain why it always returns a string.
4. **Task 4:** Use arithmetic and assignment operators in a complete program, paying attention to operator precedence.
5. **Task 5:** Write a program that converts temperatures and computes compound growth, tracing every conversion by hand.

Details of Lab Experiment:

Task 1: Core data types

```
>>> age = 21                    # int
>>> height = 5.9                # float
>>> name = "Ayesha"             # str
>>> is_enrolled = True          # bool
>>> type(age)
<class 'int'>
>>> type(height)
<class 'float'>
```

Listing 4: Inspecting types in the interpreter

Python is dynamically typed: a variable is just a label bound to a value, and the same name may legally refer to different types over its lifetime. This flexibility speeds up prototyping but means type mistakes surface at runtime rather than compile time – so get in the habit of checking `type()` when behavior surprises you.

Task 2: Type conversion

```
>>> int(3.99)          # truncates toward zero -> 3
3
>>> float(7)           # -> 7.0
7.0
>>> str(42)            # -> '42'
'42'
>>> int("123")         # parses digits -> 123
123
>>> int("12.5")        # ValueError!
```

Listing 5: Explicit conversions

Note the last case: converting a decimal string directly to `int` raises an error – you must go through `float` first. This is one of the most common beginner crashes when processing numeric user input.

Task 3: Reading input

`input()` pauses the program, echoes typed characters until Enter, and returns *everything* as a string – even if the user typed pure digits. Every numeric read therefore requires an explicit conversion step, as shown below.

Tasks 4–5: Complete programs

```
# finance_weather.py
principal = float(input("Enter principal (PKR): "))
rate      = float(input("Enter annual rate (%): "))
years     = int(input("Enter years: "))

simple_interest = principal * rate * years / 100
print(f"Simple interest after {years} yrs = {
   simple_interest:.2f}")

celsius = float(input("Enter temperature in C: "))
fahrenheit = celsius * 9 / 5 + 32
print(f"{celsius} C = {fahrenheit:.1f} F")
```

Listing 6: Simple interest and temperature converter

Trace the temperature line by hand for `37`: $37 \times 9 = 333$, $333/5 = 66.6$, $66.6 + 32 = 98.6$. Confirm the program prints `98.6`. Operator precedence follows standard mathematics (exponent, then multiply/divide, then add/subtract); when in doubt, add parentheses – they cost nothing and prevent subtle bugs.

Summary:

This week you worked with Python's fundamental types, practiced explicit conversions, and built interactive programs that validate-free-read user input and format clean output. The "input returns strings" rule will explain half of the runtime errors you encounter this semester.

3 Week 3: Control Flow – Decisions and Loops

Objective: Direct the flow of a program using conditional statements and loops, the building blocks of every algorithm you will implement this semester.

Tasks:

1. **Task 1:** Use `if`, `elif`, and `else` with comparison and logical operators, and translate a grading policy into code.
2. **Task 2:** Repeat work with a `while` loop, and identify what makes a loop terminate versus loop forever.
3. **Task 3:** Iterate with `for` and `range()`, including stepped and reversed ranges.
4. **Task 4:** Control loop execution precisely using `break` and `continue`.
5. **Task 5:** Write a number-guessing game combining loops, conditionals, and the `random` module.

Details of Lab Experiment:

Task 1: Conditionals

```
marks = float(input("Enter marks (0-100): "))

if marks >= 90:
    grade = "A+"
elif marks >= 80:
    grade = "A"
elif marks >= 70:
    grade = "B"
elif marks >= 60:
    grade = "C"
else:
    grade = "F"

print(f"Marks {marks} => Grade {grade}")
```

Listing 7: Grading policy implemented with if/elif/else

Python tests conditions top-to-bottom and runs only the first true branch, so ordering matters: checking `marks >= 60` before `marks >= 90` would assign C to every high scorer. Indentation (conventionally 4 spaces) is not

cosmetic in Python – it *defines* which statements belong to which branch, and mixing indentation levels is a syntax error. Compound conditions combine with `and`, `or`, and `not`, e.g. `if 0 <= marks <= 100` chains comparisons naturally.

Task 2: While loops

```
n = int(input("Enter a positive integer: "))
total = 0
while n > 0:
    total += n % 10       # take last digit
    n //= 10              # remove last digit
print("Digit sum =", total)
```

Listing 8: Summing digits of a number with a while loop

A `while` loop repeats as long as its condition remains true. Every loop variable must move toward termination inside the body – here `n //= 10` strictly shrinks `n` toward zero. Forgetting that update is the classic cause of infinite loops; remember Ctrl+C interrupts a runaway program in a terminal.

Task 3: For loops and range

```
num = int(input("Table of: "))
for i in range(1, 11):
    print(f"{num} x {i:2d} = {num * i:3d}")
```

Listing 9: Multiplication table using for and range

`range(start, stop, step)` generates integers from `start` up to but *not including* `stop`. `range(5)` means 0..4; `range(10, 0, -1)` counts down. This half-open convention appears throughout Python, including array slicing in Week 7, so internalize it now.

Task 4: break and continue

```
for n in range(2, 50):
    if n % 2 == 0:
        continue          # skip even numbers entirely
    if n == 1:
        break
    if n == 23:
        print("Found 23!")
        break             # exit the loop completely
```

Listing 10: Searching with break, skipping with continue

`break` abandons the nearest enclosing loop immediately; `continue` jumps straight to the next iteration, skipping the rest of the body. They express "found it – stop" and "this one doesn't count – next" more clearly than deep flag-variable logic.

Task 5: Number-guessing game

```
import random

secret = random.randint(1, 100)
tries = 0
while True:
    guess = int(input("Your guess (1-100): "))
    tries += 1
    if guess < secret:
        print("Too low!")
    elif guess > secret:
        print("Too high!")
    else:
        print(f"Correct! You took {tries} tries.")
        break
```

Listing 11: Interactive guessing game

This program composes everything from this week: an unconditional loop (`while True`) exited by `break`, branching feedback, a running counter, and pseudorandom numbers from the `random` module – whose deterministic-seed cousin (`random.seed()`) you will meet again when shuffling datasets in the machine-learning weeks.

Summary:

You can now express decision logic and repetition in Python. Every AI algorithm in this course – from gradient descent iterations to epoch loops in neural networks – is ultimately built from these same control-flow primitives.

4 Week 4: Core Data Structures – Lists, Tuples, Dictionaries, Sets

Objective: Organize collections of data using Python's built-in containers and choose the right structure for each task.

Tasks:

1. **Task 1:** Create lists, access elements by index, slice them, and mutate them with core list methods.
2. **Task 2:** Use tuples for fixed records, and explain why their immutability matters (e.g. as dictionary keys).
3. **Task 3:** Build dictionaries mapping keys to values, and iterate over keys, values, and items safely.
4. **Task 4:** Apply set operations to remove duplicates and compute unions/intersections.
5. **Task 5:** Write a student-marks manager combining all four structures, including a list comprehension.

Details of Lab Experiment:

Task 1: Lists

```
scores = [88, 92, 79, 95, 67]

print(scores[0])         # 88   (indexing starts at 0)
print(scores[-1])        # 95   (negative index = from the
    end)
print(scores[1:4])       # [92, 79, 95]

scores.append(73)        # add at the end
scores.insert(2, 85)     # insert at position 2
scores.remove(67)        # delete by value
last = scores.pop()      # delete and return last item
scores.sort()            # sort in place
print(len(scores), max(scores), sum(scores))
```

Listing 12: List creation, slicing, and mutation

Slicing `scores[1:4]` again follows the half-open rule: start index included, stop index excluded. A slice with omitted bounds copies the whole list (`scores[:]`), which is the safest way to snapshot a list before sorting it.

Task 2: List comprehensions and tuples

```
celsius = [0, 20, 37, 100]
fahr = [(c * 9 / 5) + 32 for c in celsius]      #
    comprehension

point = (3, 4)              # tuple: immutable record
x, y = point                # unpacking
print(f"x={x}, y={y}, magnitude={(x**2 + y**2)**0.5}")
```

Listing 13: Comprehensions vs explicit loops, and tuples

The comprehension is the idiomatic Python translation of "build a new list by transforming each element," and you will use it constantly for data cleaning. Tuples are immutable sequences – once created they cannot change – which makes them safe as dictionary keys and ideal for fixed-shape records like (x, y) coordinates.

Task 3: Dictionaries

```
ages = {"Ali": 20, "Sara": 22, "Ahmed": 21}

ages["Hina"] = 19                # insert
print(ages.get("Bilal", "N/A")) # safe lookup with default

for name, age in ages.items():
    print(f"{name} is {age} years old")
```

Listing 14: Dictionary operations and iteration

Dictionaries store key→value pairs with average $O(1)$ lookup. Accessing a missing key with square brackets raises `KeyError`; `.get()` lets you supply a fallback instead. Iterating with `.items()` gives both halves of each pair cleanly.

Task 4: Sets

```
>>> a = {1, 2, 3, 4}
>>> b = {3, 4, 5}
>>> a | b           # union
{1, 2, 3, 4, 5}
>>> a & b           # intersection
{3, 4}
>>> set([1, 2, 2, 3, 3, 3])   # de-duplicate a list
{1, 2, 3}
```

Listing 15: Set algebra

Sets hold unordered unique elements and support fast membership tests. De-duplicating a feature list or comparing two vocabularies (as in spam filtering) is a natural set operation.

Task 5: Student-marks manager

```
students = {
    "Ali":  [85, 90, 78],
    "Sara": [92, 88, 95],
    "Ahmed":[60, 71, 68],
}

averages = {name: sum(m)/len(m) for name, m in students.
   items()}
top_student = max(averages, key=averages.get)

print("Averages:")
for name, avg in sorted(averages.items()):
    print(f"  {name:<6} {avg:.2f}")
print(f"Top student: {top_student} ({averages[top_student
   ]:.2f})")

passed = {n for n, a in averages.items() if a >= 70}
print("Passed:", passed)
```

Listing 16: Combining containers in one program

Observe how each container plays its natural role: the outer dictionary maps names to mark-lists, a dict comprehension builds the averages, `max()` with `key=` finds the winner without a manual loop, and a set comprehension collects who passed. This composition style – short declarative transformations over containers – is exactly how real data-processing pipelines in later labs are written.

Summary:

You can now select among lists, tuples, dictionaries, and sets based on whether order, immutability, key-based lookup, or uniqueness matters. Pandas DataFrames in Week 8 generalize these containers, so fluency here pays forward directly.

5 Week 5: Functions and Modules

Objective: Structure programs into reusable, testable units using functions, parameters, return values, and library modules.

Tasks:

1. **Task 1:** Define functions with parameters and return values, and distinguish printing from returning.
2. **Task 2:** Use default arguments and keyword arguments, and explain when defaults are evaluated.
3. **Task 3:** Understand local vs global scope, and predict a variable's value across function calls.
4. **Task 4:** Import and use standard-library modules (`math`, `random`), including selective imports.
5. **Task 5:** Write a menu-driven calculator built entirely from functions, plus a dice-simulation experiment.

Details of Lab Experiment:

Tasks 1–2: Defining functions

```
def bmi(weight_kg, height_m):
    """Return Body Mass Index."""
    return weight_kg / height_m ** 2

def greet(name, greeting="Assalam-o-Alaikum"):
    """Greet with an optional custom greeting."""
    return f"{greeting}, {name}!"

print(bmi(70, 1.75))
print(greet("Ali"))                          # uses default
print(greet("Ali", greeting="Hello"))        # keyword
    argument
result = bmi(80, 1.8)                        # returned
    value captured
```

Listing 17: Parameters, returns, and defaults

A function that only `prints` cannot be reused in further computation; returning a value makes it composable. Default arguments are evaluated once, at definition time – a subtlety that becomes important when the default is a mutable object like a list.

Task 3: Scope

```
count = 0                          # global

def increment(n):
    total = n + 1                  # local: exists only inside
    this call
    return total

increment(5)
print(count)                       # still 0
print(total)                       # NameError: total is gone
```

Listing 18: Local vs global scope

Names assigned inside a function are local and vanish when the call ends; the function reads outer names freely but should communicate results back through `return` rather than mutating globals – a discipline that keeps larger AI scripts debuggable.

Task 4: Modules

```
>>> import math
>>> math.sqrt(2)
1.4142135623730951
>>> from random import randint, choice
>>> randint(1, 6)
4
>>> choice(["rock", "paper", "scissors"])
'paper'
```

Listing 19: Import styles

Plain `import` keeps the module namespace (`math.sqrt`); `from ... import` pulls specific names in directly. In coming weeks you will routinely write `import numpy as np` and `import pandas as pd`, extending exactly this mechanism to scientific libraries.

Task 5: Calculator and dice experiment

```
def add(a, b):      return a + b
def subtract(a, b): return a - b
def multiply(a, b): return a * b
def divide(a, b):
    if b == 0:
        return None            # guard against divide-by-
    zero
    return a / b
```

```
def main():
    while True:
        print("\n1:Add  2:Sub  3:Mul  4:Div  5:Quit")
        ch = input("Choice: ")
        if ch == "5":
            break
        x = float(input("First number: "))
        y = float(input("Second number: "))
        ops = {"1": add, "2": subtract, "3": multiply, "4"
   : divide}
        result = ops[ch](x, y)
        print("Result:", result if result is not None
                          else "Cannot divide by zero")

main()
```

Listing 20: Menu-driven calculator using functions

The dispatch dictionary maps choices to function objects – functions are values in Python and can be stored, passed, and returned. Note also the explicit division-by-zero guard: unlike the 8086's hardware exception, Python raises a catchable `ZeroDivisionError`, but preventing it outright is cleaner.

```
from random import randint
from collections import Counter

rolls = [randint(1, 6) for _ in range(6000)]
counts = Counter(rolls)
for face in range(1, 7):
    print(f"Face {face}: {counts[face]:5d}"
          f" ({counts[face]/6000:.1%})")
```

Listing 21: Dice-roll simulation – preview of statistics

With 6000 rolls each face should approach ≈16.7%. This empirical check of theoretical probability is your first taste of Monte-Carlo experimentation – the same technique later underlies train/test splitting and stochastic gradient descent.

Summary:

You can now decompose problems into functions with clear inputs and outputs, control scope deliberately, and leverage standard-library modules. From next week onward, virtually every lab consists of gluing powerful third-party modules together with exactly these skills.

6 Week 6: Strings and File Handling

Objective: Process text thoroughly and persist data to disk – prerequisites for cleaning datasets and logging experiments.

Tasks:

1. **Task 1:** Apply core string methods (search, replace, strip, split, join) and case conversions.
2. **Task 2:** Slice and index strings, reverse them, and test membership with `in`.
3. **Task 3:** Write text files with `open()` context managers, distinguishing write from append mode.
4. **Task 4:** Read files back line-by-line and parse their contents into structured Python data.
5. **Task 5:** Write a word-frequency analyzer over a text file, producing alphabetically sorted output.

Details of Lab Experiment:

Tasks 1–2: String methods

```
>>> s = "  Artificial Intelligence  "
>>> s.strip()
'Artificial Intelligence'
>>> s.strip().lower().replace(" ", "_")
'artificial_intelligence'
>>> "data" in "data science"
True
>>> "abcdef"[::-1]           # slice-step trick reverses
'fedcba'
>>> "17,25,8".split(",")
['17', '25', '8']
>>> "-".join(["a", "b", "c"])
'a-b-c'
```

Listing 22: Essential string operations

Strings are immutable – every method returns a *new* string, so chaining as above is safe and idiomatic. Method chaining for cleaning raw text (trim, normalize case, fix separators) is exactly the pattern used to prepare text corpora in NLP labs.

Tasks 3–4: Writing and reading files

```
# --- write ---
with open("experiment_log.txt", "w") as f:
    f.write("epoch,loss\n")
    for epoch in range(1, 6):
        loss = 1.0 / epoch
        f.write(f"{epoch},{loss:.4f}\n")

# --- append ---
with open("experiment_log.txt", "a") as f:
    f.write("# training finished\n")

# --- read back line by line ---
with open("experiment_log.txt") as f:
    for line in f:
        line = line.strip()
        if not line or line.startswith("#"):
            continue
        epoch, loss = line.split(",")
        print(f"epoch={epoch:>2}  loss={loss}")
```

Listing 23: Writing then reading a text file

Mode `"w"` truncates the existing file, `"a"` appends, and omitting the mode defaults to reading. The `with` block guarantees the file closes even if an exception occurs – never rely on manual `close()`. Parsing splits each line on commas, converting fields as needed; note the defensive skip of blank and comment lines, a habit that saves hours when logs contain noise.

Task 5: Word-frequency analyzer

```
import string

with open("article.txt", "w") as f:
    f.write("AI is changing the world. "
            "The world is changing faster than ever!\n")

counts = {}
with open("article.txt") as f:
    for line in f:
        line = line.lower().translate(
            str.maketrans("", "", string.punctuation))
        for word in line.split():
            counts[word] = counts.get(word, 0) + 1

for word in sorted(counts):
    print(f"{word:<10} {counts[word]}")
```

Listing 24: Counting words in a text file

The pipeline is: lowercase, strip punctuation, split on whitespace, tally with a dictionary. `str.maketrans("", "", string.punctuation)` builds a translation table deleting punctuation characters. This tiny analyzer is structurally identical to building vocabulary dictionaries in text-classification models – only the scale differs.

Summary:

You can now transform strings fluently and read/write files robustly. Combined with Week 5's functions, you have every tool needed for the tabular-data work that begins next week with NumPy and Pandas.

7 Week 7: Numerical Computing with NumPy

Objective: Manipulate multi-dimensional arrays efficiently with NumPy – the foundation layer beneath Pandas, scikit-learn, and every deep-learning framework.

Tasks:

1. **Task 1:** Create arrays from lists and with generators (`arange`, `linspace`, `zeros`), and inspect `shape` and `dtype`.
2. **Task 2:** Index and slice 2-D arrays, including row/column selection and boolean masks.
3. **Task 3:** Exploit vectorization and broadcasting to replace explicit loops with array expressions.
4. **Task 4:** Compute aggregations along axes (sum, mean, max) and understand the difference between axis 0 and axis 1.
5. **Task 5:** Implement matrix operations and a z-score standardization – both used constantly in machine learning.

Details of Lab Experiment:

Task 1: Creating arrays

```
import numpy as np

a = np.array([1, 2, 3, 4])           # from a list
m = np.arange(12).reshape(3, 4)      # 0..11 shaped 3x4
lin = np.linspace(0, 1, 5)           # 5 evenly spaced
    points
z = np.zeros((2, 3))

print(m.shape, m.dtype)              # (3, 4) int64
print(lin)                           # [0.   0.25 0.5  0.75
     1. ]
```

Listing 25: Array creation and attributes

Unlike Python lists, a NumPy array is homogeneous (one `dtype`) and stores data contiguously, enabling vectorized operations that run tens of times faster than equivalent Python loops. Reshaping never copies data unnecessarily – `reshape` merely reinterprets the same buffer, which is why it costs almost nothing.

Task 2: Indexing, slicing, masking

```
>>> m = np.arange(12).reshape(3, 4)
>>> m[1, 2]                 # single element (row 1, col 2)
6
>>> m[0, :]                 # entire row 0
array([0, 1, 2, 3])
>>> m[:, 1]                 # entire column 1
array([1, 5, 9])
>>> m[m > 7]                # boolean mask
array([ 8,  9, 10, 11])
```

Listing 26: 2-D access patterns

Boolean masking selects every element satisfying a condition in one expression – the direct ancestor of Pandas filtering next week, and of dataset subsetting in the ML weeks (e.g. `X[y == 0]` to grab all samples of class 0).

Task 3: Vectorization and broadcasting

```
prices = np.array([100, 250, 40, 75])
discounted = prices * 0.85          # scalar broadcast to
    all

row_totals = prices + np.array([10, 20, 30, 40])

temps = np.array([[15, 18, 21],
                  [22, 25, 28]])
temp_c_from_f = temps * 9 / 5 + 32  # whole-matrix formula
```

Listing 27: Loop-free mathematics

Operations apply elementwise without any explicit loop. Broadcasting extends smaller shapes against larger ones automatically – adding a 1-D bias vector to every row of a matrix is the canonical neural-network example of this behavior.

Task 4: Aggregations and axes

```
>>> m = np.arange(12).reshape(3, 4)
>>> m.sum()           # grand total
66
>>> m.sum(axis=0)     # collapse rows -> per-column totals
array([12, 15, 18, 21])
>>> m.mean(axis=1)    # collapse columns -> per-row means
array([1.5, 5.5, 9.5])
```

Listing 28: Axis semantics

Memorize the convention: `axis=0` travels *down* the rows (producing one value per column), `axis=1` travels *across* the columns (one value per row). Nearly every aggregation error beginners make is simply the wrong axis.

Task 5: Matrix operations and standardization

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
print(A @ B)                      # matrix multiplication
print(np.linalg.inv(A))           # matrix inverse

np.random.seed(42)
raw = np.random.randint(50, 101, size=(5, 3))   # fake
    marks

mean = raw.mean(axis=0)
std  = raw.std(axis=0)
standardized = (raw - mean) / std               # z-scores
print(np.round(standardized, 2))
print(standardized.mean(axis=0).round(2))       # ~0 per
    column
print(standardized.std(axis=0).round(2))        # ~1 per
    column
```

Listing 29: Dot products, inverse, and z-score normalization

The `@` operator performs true matrix multiplication – the operation underlying every layer of every neural network. The z-score transform $(x - \mu)/\sigma$ rescales each column to mean 0 and standard deviation 1; verify both printed checks. Feature scaling in exactly this form is applied to datasets in Week 11 before training, because many learning algorithms misbehave when features live on wildly different scales.

Summary:

You can now create, index, mask, broadcast, and aggregate NumPy arrays, and implement two staples of applied ML: matrix products and standardization. Pandas next week adds labeled columns and missing-data handling on top of these exact array mechanics.

8 Week 8: Data Analysis with Pandas

Objective: Load, clean, explore, and summarize tabular datasets – the daily workflow of every applied AI practitioner.

Tasks:

1. **Task 1:** Create Series and DataFrames from Python structures and read a CSV file from disk.
2. **Task 2:** Inspect a dataset with `head`, `info`, `describe`, and `shape`.
3. **Task 3:** Select data by label and position (`loc`/`iloc`) and filter rows with boolean conditions.
4. **Task 4:** Detect and handle missing values, and derive new columns from existing ones.
5. **Task 5:** Summarize a student dataset with `groupby` aggregations and sorting.

Details of Lab Experiment:

Tasks 1–2: Creating and inspecting DataFrames

```
import pandas as pd

data = {
    "Name":   ["Ali", "Sara", "Ahmed", "Hina", "Bilal"],
    "Gender": ["M", "F", "M", "F", "M"],
    "Math":   [85, 92, 60, 78, None],
    "Physics":[90, 88, 71, None, 65],
    "CS":     [78, 95, 68, 82, 74],
}
df = pd.DataFrame(data)
df.to_csv("students.csv", index=False)

loaded = pd.read_csv("students.csv")
print(loaded.head())          # first five rows
print(loaded.info())          # dtypes + non-null counts
print(loaded.describe())      # count/mean/std/quartiles
print(loaded.shape)           # (rows, cols)
```

Listing 30: Building a dataset and saving/loading CSV

`info()` is your first diagnostic on any new dataset: it reveals column types and exactly which columns contain missing values. `describe()` gives instant summary statistics for all numeric columns. We deliberately planted two `None` entries to exercise Task 4.

Task 3: Selection and filtering

```
df = pd.read_csv("students.csv")

print(df.loc[0])                    # row by label
print(df.iloc[0, 2])                # row 0, col 2 by
    position
print(df.loc[:, ["Name", "CS"]]) # column subset

high_cs = df[df["CS"] > 80]                         # condition
girls = df[(df["Gender"] == "F") &
           (df["Math"] > 75)]                       # combined
    AND
print(high_cs, girls, sep="\n\n")
```

Listing 31: loc, iloc, and boolean filtering

Inside the brackets, a boolean Series aligns with the DataFrame's rows and keeps only `True` positions – vectorized filtering with zero explicit looping. Conditions combine with `&`/`|` (never `and`/`or`), and each condition must be parenthesized because of operator precedence.

Task 4: Missing values and derived columns

```
df = pd.read_csv("students.csv")

print(df.isnull().sum())            # NaN count per column
filled = df.fillna(df[["Math", "Physics"]].mean().round(1)
    )
dropped = df.dropna()               # alternative: lose
    rows

filled["Total"] = filled[["Math", "Physics", "CS"]].sum(
    axis=1)
filled["Percentage"] = (filled["Total"] / 300 * 100).round
    (2)
print(filled.sort_values("Percentage", ascending=False))
```

Listing 32: Handling NaN and engineering features

Two standard strategies exist: impute missing entries (`fillna` with a statistic – here the column mean) or discard incomplete rows (`dropna`). Which is right depends on how much data you can afford to lose; imputation preserves sample count and is the usual default in ML preprocessing. Deriving `Total` and `Percentage` demonstrates feature construction – creating informative columns from raw ones.

Task 5: GroupBy summaries

```
df = pd.read_csv("students.csv").dropna()

summary = df.groupby("Gender")[["Math", "CS"]].mean().
    round(2)
print(summary)

print(df.groupby("Gender")["Name"].count())
print(df.nlargest(3, "CS")[["Name", "CS"]])    # top-3 in
    CS
```

Listing 33: Split-apply-combine with groupby

`groupby` implements split-apply-combine: split rows into groups by key, apply an aggregation per group, combine the results into a new frame. This single idiom answers most exploratory questions ("average CS score by gender") in one line – and its mental model carries over directly to SQL's `GROUP BY` and to pivot tables.

Summary:

You completed the canonical Pandas cycle – load, inspect, filter, clean, engineer, aggregate – on a realistic dataset. Every dataset you touch for the rest of this course passes through exactly these steps before any model sees it.

9 Week 9: Data Visualization with Matplotlib and Seaborn

Objective: Produce clear, labeled charts that expose distributions, relationships, and trends in data.

Tasks:

1. **Task 1:** Draw and customize a line plot with axis labels, title, legend, grid, and markers.
2. **Task 2:** Compare categories with bar charts, and examine distributions with histograms.
3. **Task 3:** Reveal correlations with scatter plots, plotting multiple series on one axes.
4. **Task 4:** Compose multiple charts in one figure using subplots.
5. **Task 5:** Visualize the Week-8 student dataset end-to-end, including a Seaborn correlation heatmap.

Details of Lab Experiment:

Task 1: Line plots

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 2 * np.pi, 200)
plt.plot(x, np.sin(x), label="sin(x)", linewidth=2)
plt.plot(x, np.cos(x), label="cos(x)", linestyle="--")
plt.xlabel("x (radians)")
plt.ylabel("value")
plt.title("Trigonometric functions")
plt.legend()
plt.grid(alpha=0.3)
plt.savefig("trig.png", dpi=150)    # save before show
plt.show()
```

Listing 34: Customized line plot of a sine wave

Every chart should be self-explanatory to someone who has not seen your code – labels, title, and legend are mandatory, not decorative. Call `savefig` *before* `show`, because `show` clears the figure buffer in some backends. Saved figures feed directly into lab reports.

Tasks 2–3: Bars, histograms, scatter

```
subjects = ["Math", "Physics", "CS"]
averages = [78.8, 78.0, 79.4]
plt.bar(subjects, averages, color=["navy", "teal", "orange
    "])
plt.ylabel("Average marks")
plt.title("Subject averages")
plt.show()

np.random.seed(0)
exam = np.random.normal(loc=70, scale=10, size=500)
plt.hist(exam, bins=20, edgecolor="black")
plt.xlabel("Marks"); plt.ylabel("Students")
plt.title("Exam distribution")
plt.show()

hours = np.random.uniform(1, 8, 100)
score = 25 + 7 * hours + np.random.normal(0, 6, 100)
plt.scatter(hours, score, alpha=0.6)
plt.xlabel("Study hours"); plt.ylabel("Score")
plt.title("Hours vs Score")
plt.show()
```

Listing 35: Bar chart, histogram, and grouped scatter

Histogram bin count controls smoothness: too few bins hide shape, too many amplify noise – always try several. The scatter shows a noisy linear relationship, the visual signature of the regression problem you will actually fit in Week 11.

Task 4: Subplots

```
fig, axes = plt.subplots(2, 2, figsize=(10, 7))
axes[0, 0].plot(x, np.sin(x));            axes[0, 0].
    set_title("Line")
axes[0, 1].bar(subjects, averages);       axes[0, 1].
    set_title("Bar")
axes[1, 0].hist(exam, bins=15);           axes[1, 0].
    set_title("Histogram")
axes[1, 1].scatter(hours, score, s=10);   axes[1, 1].
    set_title("Scatter")
plt.tight_layout()
plt.show()
```

Listing 36: A 2x2 grid of charts

The object-oriented interface (`axes[i, j]`) scales far better than repeated calls to global state once figures contain multiple panels – prefer it whenever a figure needs more than one plot.

Task 5: Dataset visualization with Seaborn

```
import pandas as pd
import seaborn as sns

df = pd.read_csv("students.csv").fillna(df[["Math","
    Physics"]].mean().round(1))
df["Total"] = df[["Math", "Physics", "CS"]].sum(axis=1)

corr = df[["Math", "Physics", "CS", "Total"]].corr()
sns.heatmap(corr, annot=True, cmap="coolwarm", vmin=-1,
    vmax=1)
plt.title("Correlation matrix")
plt.tight_layout(); plt.show()

sns.barplot(data=df, x="Gender", y="Total")
plt.title("Average total by gender"); plt.show()
\end{lstreading}
The diagonal of a correlation matrix is always 1; strong
    off-diagonal cells flag redundant or predictive
    features -- a genuine feature-selection insight, not
    just a picture. Seaborn's \texttt{barplot}
    automatically aggregates and draws confidence intervals
    , doing in one call what required manual \texttt{
    groupby} plumbing last week.
```

Listing 37: Correlation heatmap and grouped bars for students.csv

Summary:

You can now turn any DataFrame into publication-quality visuals and, more importantly, *read* charts to form hypotheses about data. Model-building begins next, and every serious modeling decision in Weeks 11–13 starts with plots exactly like these.

10 Week 10: Object-Oriented Programming

Objective: Model real-world entities as classes, and organize larger AI projects around objects with state and behavior.

Tasks:

1. **Task 1:** Define a class with `__init__`, instance attributes, and methods; instantiate multiple objects.
2. **Task 2:** Distinguish instance attributes from class attributes, and explain when each is appropriate.
3. **Task 3:** Apply encapsulation using name-mangled private attributes with getter-style methods.
4. **Task 4:** Derive subclasses through inheritance and extend or override parent behavior.
5. **Task 5:** Implement a bank-account system and a pair of related classes modeling ML datasets.

Details of Lab Experiment:

Tasks 1–2: Classes, objects, and attributes

```
class Student:
    institution = "NFC IET Multan"      # class attribute (
    shared)

    def __init__(self, name, roll_no):
        self.name = name                # instance
    attributes
        self.roll_no = roll_no
        self.marks = []

    def add_mark(self, m):
        self.marks.append(m)

    def average(self):
        return sum(self.marks) / len(self.marks) if self.
    marks else 0.0

    def __str__(self):
        return f"{self.name} ({self.roll_no}) avg={self.
    average():.1f}"

s1 = Student("Ali", "iet-01")
```

```
s2 = Student("Sara", "iet-02")
s1.add_mark(85); s1.add_mark(90)
print(s1)                          # __str__ invoked by print
print(Student.institution)         # shared by all instances
```

Listing 38: A minimal class and its instances

`__init__` runs at instantiation and `self` is the concrete object being constructed – the first parameter of every method, passed implicitly. Class attributes belong to the class itself and are shared; instance attributes live per-object. When you later call `model.fit(X, y)`, you are invoking a method on an object holding weights, hyperparameters, and history – exactly this pattern at industrial scale.

Task 3: Encapsulation

```
class Temperature:
    def __init__(self, celsius):
        self.__celsius = celsius        # name-mangled "
   private"

    def get_celsius(self):
        return self.__celsius

    def set_celsius(self, value):
        if value < -273.15:
            raise ValueError("Below absolute zero!")
        self.__celsius = value

t = Temperature(25)
t.set_celsius(30)
print(t.get_celsius())
```

Listing 39: Protecting internal state

The double-underscore prefix triggers name mangling, blocking accidental external modification. Validation lives in the setter, so the object can never enter an invalid state – a principle worth copying whenever a class guards something critical (currency amounts, physical limits, model hyperparameters).

Task 4: Inheritance

```
class Person:
    def __init__(self, name):
        self.name = name
    def introduce(self):
        return f"I am {self.name}"
```

```
class Teacher(Person):
    def __init__(self, name, subject):
        super().__init__(name)          # reuse parent
    setup
        self.subject = subject
    def introduce(self):                # override
        return f"I am {self.name}, teaching {self.subject}
    "

people = [Person("Ahmed"), Teacher("Dr. Siddique", "AI")]
for p in people:
    print(p.introduce())
```

Listing 40: Extending a base class

`super().__init__` delegates shared initialization upward instead of duplicating it. Overriding `introduce` specializes behavior while the loop treats every object uniformly – polymorphism in action, and precisely how scikit-learn exposes one uniform `fit`/`predict` API across dozens of different model classes.

Task 5: Integrated exercises

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.__balance = balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Deposit must be positive")
        self.__balance += amount

    def withdraw(self, amount):
        if amount > self.__balance:
            raise ValueError("Insufficient funds")
        self.__balance -= amount

    @property
    def balance(self):
        return self.__balance

acct = BankAccount("Ali", 1000)
acct.deposit(500); acct.withdraw(300)
print(acct.balance)          # 1200
```

Listing 41: BankAccount with guarded transactions

The `@property` decorator exposes the private balance read-only, so ex-

ternal code can inspect but not corrupt it. As a second exercise, design `DatasetLoader` with a `load()` method returning a list of records, then subclass it as `CSVDatasetLoader` overriding only the parsing step – mirroring how real ML frameworks abstract storage formats behind a common loader interface.

Summary:

You can now design classes with protected state, controlled interfaces, and inheritance hierarchies. Next week's scikit-learn estimators will feel familiar: they are objects you construct, configure via attributes, and drive through methods.

11 Week 11: Machine Learning Fundamentals with scikit-learn

Objective: Train, evaluate, and compare supervised learning models using the scikit-learn workflow.

Tasks:

1. **Task 1:** Explain the supervised-learning setup – features, labels, train/test split – and why held-out data is non-negotiable.
2. **Task 2:** Fit a linear regression model on synthetic data and interpret its learned coefficients.
3. **Task 3:** Evaluate regression predictions with MAE and R^2, and visualize residuals.
4. **Task 4:** Train a k-NN classifier on the Iris dataset and measure accuracy with a confusion matrix.
5. **Task 5:** Standardize features within a proper Pipeline and compare classifiers fairly.

Details of Lab Experiment:

Tasks 1–2: Linear regression

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

rng = np.random.RandomState(42)
hours = rng.uniform(1, 9, 120).reshape(-1, 1)    # 2-D
    features
scores = 25 + 6.5 * hours.ravel() \
         + rng.normal(0, 5, 120)                  # noisy
    truth

X_train, X_test, y_train, y_test = train_test_split(
    hours, scores, test_size=0.2, random_state=42)

model = LinearRegression().fit(X_train, y_train)
print(f"Learned: score = {model.intercept_:.2f}"
      f" + {model.coef_[0]:.2f} * hours")
print(f"Test R^2: {model.score(X_test, y_test):.3f}")
```

Listing 42: Fitting a line to study-hours data

The estimator API is uniformly three beats: construct, `fit`, `predict`/`score`. Features must be 2-D (`reshape(-1, 1)`) hence the explicit reshape. The recovered coefficients (≈ 25 and ≈ 6.5) match the generating process we planted – regression has essentially re-discovered the hidden law from noisy samples. Evaluating on the test split, which the model never saw during fitting, is what separates honest assessment from wishful thinking.

Task 3: Regression diagnostics

```
from sklearn.metrics import mean_absolute_error, r2_score

preds = model.predict(X_test)
mae = mean_absolute_error(y_test, preds)
r2  = r2_score(y_test, preds)
print(f"MAE = {mae:.2f} marks, R^2 = {r2:.3f}")

residuals = y_test - preds
plt.scatter(preds, residuals, alpha=0.7)
plt.axhline(0, color="red", linestyle="--")
plt.xlabel("Predicted"); plt.ylabel("Residual")
plt.title("Residual analysis"); plt.show()
```

Listing 43: MAE, R-squared, and residual plot

MAE speaks in target units ("off by about 4 marks"), while R^2 measures variance explained (1.0 is perfect, 0 is no better than predicting the mean). Residuals scattered symmetrically around zero indicate a healthy fit; visible structure (curves, funnels) signals model misspecification or heteroscedastic noise.

Task 4: Classification on Iris

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix,
    accuracy_score

iris = load_iris(as_frame=True)
X, y = iris.data, iris.target

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=1)

clf = KNeighborsClassifier(n_neighbors=5).fit(X_tr, y_tr)
preds = clf.predict(X_te)

print("Accuracy:", accuracy_score(y_te, preds))
```

```
print(pd.DataFrame(confusion_matrix(y_te, preds),
                   index=iris.target_names,
                   columns=iris.target_names))
```

Listing 44: k-NN classification with confusion matrix

k-NN classifies a flower by majority vote among its five nearest training samples in feature space. `stratify=y` preserves class proportions in both splits – important whenever classes are imbalanced. The confusion matrix shows *where* mistakes happen (here typically a few `versicolor`↔`virginica` confusions, since those species overlap), which raw accuracy alone conceals.

Task 5: Pipelines and scaling done right

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

models = {
    "k-NN":  KNeighborsClassifier(5),
    "SVC":   make_pipeline(StandardScaler(), SVC()),
}

for name, m in models.items():
    m.fit(X_tr, y_tr)
    print(f"{name}: {accuracy_score(y_te, m.predict(X_te))
    :.3f}")
```

Listing 45: Scaling inside a Pipeline to avoid leakage

Distance-based methods like k-NN and margin-based methods like SVC are sensitive to feature scales, so standardization helps – but the scaler's statistics must come from *training data only*. Wrapping the scaler and model in a `Pipeline` guarantees this: `fit` computes scaling on the training fold alone and applies it consistently everywhere, eliminating the subtle test-set leakage that inflates naive results.

Summary:

You executed the complete supervised-learning loop – split, fit, evaluate, diagnose, improve – twice: once for regression, once for classification. Weeks 12 and 13 reuse this identical discipline with neural networks.

12 Week 12: Neural Networks with Keras/TensorFlow

Objective: Build, train, and evaluate a feed-forward neural network, and connect its machinery to concepts from previous weeks.

Tasks:

1. **Task 1:** Describe a neuron, layers, weights, and activation functions; relate matrix multiplication (Week 7) to a layer computation.
2. **Task 2:** Load and preprocess the MNIST handwriting dataset, justifying the pixel-scaling step.
3. **Task 3:** Construct a Sequential model with Dense layers and choose correct output-layer settings for 10-class classification.
4. **Task 4:** Compile and train the network, monitoring training vs validation accuracy across epochs.
5. **Task 5:** Evaluate on unseen test data, visualize sample predictions, and tune one hyperparameter.

Details of Lab Experiment:

Task 1: From neuron to network

Each Dense layer computes $\mathbf{a} = f(W\mathbf{x} + \mathbf{b})$: a matrix product and bias shift followed by an elementwise activation f (commonly `ReLU`, $\max(0, x)$). Stacking layers composes nonlinear transforms, giving the network the capacity to approximate complex decision boundaries that a single linear layer – essentially Week 11's regression – cannot represent. Training adjusts W and $\mathbf{b}$ iteratively via gradient descent to minimize a loss function; each pass over the full dataset is one *epoch*.

Task 2: MNIST preprocessing

```
import numpy as np
from tensorflow import keras

(X_train, y_train), (X_test, y_test) = \
    keras.datasets.mnist.load_data()

print(X_train.shape, X_test.shape)   # (60000,28,28)
    (10000,28,28)

X_train = X_train.astype("float32") / 255.0   # scale
    0-255 -> 0-1
```

```
X_test  = X_test.astype("float32") / 255.0
```

Listing 46: Loading and preparing MNIST

Pixels arrive as integers 0–255; dividing by 255 confines them to $[0, 1]$. Networks train poorly on large unnormalized magnitudes because gradients become unstable – the same reasoning as Week 11's `StandardScaler`, applied here by hand since images have no named columns.

Tasks 3–4: Building and training

```
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),   # 784
    inputs
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dense(64,  activation="relu"),
    keras.layers.Dense(10,  activation="softmax"),
])

model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

history = model.fit(X_train, y_train,
                    epochs=10, batch_size=32,
                    validation_split=0.1, verbose=2)
```

Listing 47: Sequential dense network trained on MNIST

`Flatten` reshapes each image to a 784-vector – the bridge between images and ordinary feature vectors. The output layer has exactly 10 neurons (one per digit) with `softmax`, producing a probability distribution summing to 1. `sparse_categorical_crossentropy` pairs with integer labels (`sparse`); had labels been one-hot encoded you would drop the prefix. `validation_split` carves 10% off the training data to monitor generalization *during* training – watch whether validation accuracy tracks training accuracy or falls behind (the signature of overfitting, which limiting epochs combats).

Task 5: Evaluation and prediction inspection

```
test_loss, test_acc = model.evaluate(X_test, y_test,
    verbose=0)
print(f"Test accuracy: {test_acc:.4f}")

probs = model.predict(X_test)
preds = probs.argmax(axis=1)
```

```
miscl = np.where(preds != y_test)[0][:10]
fig, axes = plt.subplots(2, 5, figsize=(12, 5))
for ax, i in zip(axes.ravel(), miscl):
    ax.imshow(X_test[i], cmap="gray")
    ax.set_title(f"true={y_test[i]} pred={preds[i]}")
    ax.axis("off")
plt.tight_layout(); plt.show()
```

Listing 48: Test evaluation and prediction gallery

Expect test accuracy around 97–98% – respectable yet clearly imperfect, and inspecting the misclassified thumbnails is instructive: many are genuinely ambiguous even to humans. As the hyperparameter exercise, retrain with the hidden sizes halved and doubled and with `epochs=3` versus `epochs=30`; record test accuracy each time and explain the capacity-vs-overfitting trade-off you observe, using the training/validation curves as evidence.

Summary:

You trained a real neural network end to end and connected every moving part – normalization, architecture, loss, epochs, validation – to foundations laid in Weeks 7–11. Deep learning is this same loop scaled up; the ideas, not the size, are what you now possess.

13 Week 13: Final Project – An End-to-End AI Application

Objective: Integrate the entire semester – Python, data handling, visualization, classical ML, and neural networks – into one documented, tested, working application, and adopt professional coding practices.

Tasks:

1. **Task 1:** Select and scope a project (e.g. student-score predictor, iris-flower classifier app, digit recognizer GUI), and write a one-paragraph specification before coding.
2. **Task 2:** Design the pipeline as separate stages – load, clean, model, evaluate, visualize – each implemented as a function or class.
3. **Task 3:** Implement the pipeline, and justify every modeling decision in comments.
4. **Task 4:** Test edge cases (missing values, out-of-range inputs, empty files) deliberately, not just the happy path.
5. **Task 5:** Apply professional practices – PEP 8 naming, docstrings, a virtual environment with pinned dependencies – and submit a runnable, documented repository.

Details of Lab Experiment:

Tasks 1–3: Reference implementation – student score predictor

```
"""Student Score Predictor.

Predicts final CS scores from Math/Physics marks using
linear regression, with a saved diagnostic figure.
"""
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score


def load_data(path):
    """Read CSV and impute missing numerics with column
   means."""
    df = pd.read_csv(path)
```

```
    num_cols = df.select_dtypes(include=np.number).columns
    df[num_cols] = df[num_cols].fillna(df[num_cols].mean()
   )
    return df


def train_model(df):
    """Fit linear regression: CS ~ Math + Physics."""
    X = df[["Math", "Physics"]]
    y = df["CS"]
    X_tr, X_te, y_tr, y_te = train_test_split(
        X, y, test_size=0.2, random_state=42)
    model = LinearRegression().fit(X_tr, y_tr)
    preds = model.predict(X_te)
    print(f"MAE={mean_absolute_error(y_te, preds):.2f}  "
          f"R^2={r2_score(y_te, preds):.3f}")
    return model, X_te, y_te, preds


def visualize(model, X_te, y_te, preds):
    """Scatter actual vs predicted with the ideal line."""
    plt.scatter(y_te, preds, alpha=0.7)
    lims = [min(y_te.min(), preds.min()),
            max(y_te.max(), preds.max())]
    plt.plot(lims, lims, "r--", label="perfect")
    plt.xlabel("Actual CS"); plt.ylabel("Predicted CS")
    plt.title("Student Score Predictor"); plt.legend()
    plt.savefig("results.png", dpi=150)


if __name__ == "__main__":
    df = load_data("students.csv")
    model, X_te, y_te, preds = train_model(df)
    visualize(model, X_te, y_te, preds)
    new_student = pd.DataFrame({"Math": [81], "Physics":
   [77]})
    print(f"Predicted CS: {model.predict(new_student)
   [0]:.1f}")
```

Listing 49: Complete pipeline: load -¿ clean -¿ train -¿ evaluate -¿ visualize

Run this against Week 8's `students.csv`. Note the architecture: three single-purpose functions plus a guarded entry point – the Week-5/10 structure at project scale. Any stage (swap regressor, change imputation) edits exactly one function. Extend the project by replacing `LinearRegression` with a `RandomForestRegressor` and comparing metrics; or wrap the digit classifier from Week 12 in a function accepting an image path.

Task 4: Testing edge cases

Exercise each failure mode explicitly and confirm the program responds sensibly rather than crashing mid-pipeline:

```
- Empty CSV / wrong filename        -> clear
   FileNotFoundError message?
- Non-numeric junk in a mark column -> caught by fillna/
   astype strategy?
- New student with Math=999         -> validated or
   garbage predicted?
- Dataset with a single row         -> does
   train_test_split fail loudly?
```

Listing 50: Edge-case checklist to verify by hand

Deliberately probing boundaries – not just rerunning the happy path – is what distinguishes software that merely seems to work from software you can trust; the same habit was emphasized in the calculator labs of every serious programming course, and it matters doubly in AI systems where silent bad predictions are harder to notice than loud crashes.

Task 5: Professional practices

- Follow PEP 8: `snake_case` for functions/variables, `CapWords` for classes, 4-space indents – consistency is the point.
- Give every public function a docstring stating purpose, parameters, and return value, as modeled in the reference implementation.
- Isolate project dependencies: `python -m venv venv`, activate it, and record exact versions with `pip freeze > requirements.txt` so your project reproduces on another machine.
- Keep notebooks for exploration but export final logic into `.py` modules importable and testable like any other code.
- Comment *why*, not *what*: the code already says what it does; future-you needs the rationale.

Summary:

In this final week you combined every semester skill – Python fluency, container manipulation, functions and classes, NumPy/Pandas data handling, visualization, and scikit-learn/Keras modeling – into a single coherent, tested, documented application. Congratulations on completing the full journey from "Hello, World!" to a working AI pipeline: the same staged

discipline you practiced here – load, clean, model, evaluate, communicate – is precisely how professional machine-learning systems are built.